

What is dbt?

dbt transforms raw warehouse data into trusted data products. You write simple SQL select statements, and dbt handles the heavy lifting by creating modular, maintainable data models that power analytics, operations, and AI – replacing the need for complex and fragile transformation code.

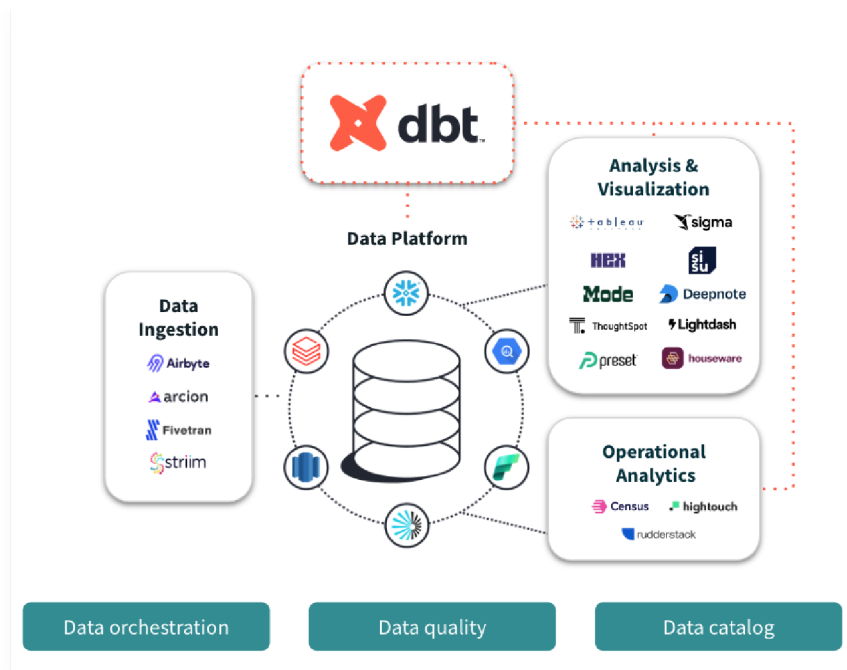
dbt is the industry standard for data transformation, helping teams work faster and produce higher-quality data. As you build in dbt, your project creates structured context — lineage, tests, contracts, metrics, and governance — that explains how your data connects, what it means, and what changes may affect.

That context makes dbt especially powerful for AI and comes with features like [dbt Wizard](#), which helps you investigate, build, validate, and ship with full project context and governance on by default.

You can use dbt and its [framework](#) to:

- Centralize and modularize your analytics code, while also providing your data team with guardrails typically found in software engineering workflows.
- Collaborate on data models to safely deploy and monitor data transformations in production.
- Apply software engineering best practices like version control, testing, modularity, CI/CD, and documentation to analytics workflows.

Backed by a 100,000+ member [community](#), dbt helps teams build high-quality, trustworthy data pipelines faster.



dbt works alongside your ingestion, visualization, and other data tools, so you can transform data directly in your cloud data platform.

Read more about why we want to enable analysts to work more like software engineers in [The dbt Viewpoint](#). Learn how other data practitioners around the world are using dbt by [joining the dbt Community](#).

dbt framework

Use the dbt framework to quickly and collaboratively transform data and deploy analytics code following software engineering best practices like version control, modularity, portability, CI/CD, and documentation. This means anyone on the data team familiar with SQL can safely contribute to production-grade data pipelines.

The dbt framework is composed of a *language* and an *engine*:

- The *dbt language* is the code you write in your dbt project — SQL select statements, Jinja templating, YAML configs, tests, and more. It's the standard for the data industry and the foundation of the dbt framework.
- The *dbt engine* compiles your project, executes your transformation graph, and produces metadata. dbt supports two engines which you can use depending on your needs:
 - The dbt Core engine, which renders Jinja and runs your models.

- The dbt Fusion engine, which goes beyond Jinja rendering to statically analyze your SQL — validating syntax and logic before your SQL is sent to the database (saving compute resources), and supports LSP features.

dbt Fusion engine

The dbt Fusion engine is a Rust-based engine that delivers a lightning-fast development experience, intelligent cost savings, and improved governance. Fusion understands SQL natively across multiple dialects, catches errors instantly, and optimizes how your models are built — bringing SQL comprehension and state awareness, instant feedback, LSP, and more to every dbt workflow.

Fusion powers dbt in the [dbt platform](#), [VS Code / Cursor](#), and [locally from the command line](#). You don't need to have a dbt platform project to use the dbt Fusion engine.

For more information, refer to [About the dbt Fusion engine](#), [supported features](#), and the [get started with Fusion](#) pages.

dbt Core engine

[dbt Core v1](#) is the open-source, Python-based engine that enables data practitioners to transform data. dbt Core v1 surfaces feedback when you run or build your project. It doesn't include Fusion features like the LSP, for example, which provides instant feedback as you type.

[dbt Core v2](#) is the open-source foundation that the dbt Fusion engine builds on. It delivers a faster, Rust-based runtime while preserving the dbt experience practitioners already know. It is currently in alpha.

Learn more with the [quickstart for dbt Core](#).

How to use dbt

You can use dbt in different ways depending on your needs:

- Using the [dbt platform](#) (recommended for most users)

- [Locally from your command line or code editor](#)
- [With dbt Wizard](#) for agentic governed data development in dbt All options support using the dbt Fusion engine or dbt Core engine.

dbt platform

The dbt platform offers the fastest, most reliable, and scalable way to deploy dbt. It can be powered by the dbt Fusion engine or dbt Core engine, and provides a fully managed service with scheduling, CI/CD, documentation hosting, monitoring, development, and alerting through a web-based user interface (UI).

The dbt platform offers [multiple ways](#) to develop and collaborate on dbt projects:

- [Develop in your browser using the Studio IDE](#)
- [Seamless drag-and-drop development with Canvas](#)
- [Run dbt commands from your local command line](#) using the dbt VS Code extension or dbt CLI (both which integrate seamlessly with the dbt platform project(s)).

Learn more about the [dbt platform features](#) and try one of the [dbt Quickstarts](#).

You can learn about plans and pricing on www.getdbt.com.

dbt local development

Use the dbt framework and develop dbt projects from your command line or code editor:

- [Install the dbt VS Code extension](#) — Combines the dbt Fusion engine performance with visual features like autocomplete, inline errors, and lineage. Includes [LSP features](#) and suitable for users with dbt platform projects or running dbt locally without a dbt platform project. *Recommended for local development.*
- [Install the Fusion CLI](#) — The dbt Fusion engine from the command line, but doesn't include LSP features.
- [Install the dbt CLI](#) — The dbt platform CLI, which allows you to run dbt commands against your dbt platform development environment from your local command line.
- [Install dbt Core](#) — The open-source, Python-based CLI that uses the dbt Core v1 engine. Doesn't include LSP features.

dbt Wizard

dbt Wizard is an AI agent purpose-built for governed data development in dbt — not just code generation, but the entire development lifecycle: investigating, building, validating, and shipping.

It's grounded in your dbt project's structured context through a [native metadata engine](#) that gives the agent high-precision access to your full project graph, including lineage, tests, contracts, and metric definitions. The context is like a map of your city: dbt Wizard knows how everything connects before it starts, rather than walking every street to figure out the layout.

dbt Wizard is different from coding agents in that it *validates* its own work against your project before you see the final diff, coordinates multi-file changes (rename a model and the refs follow), and ships with governance and audit trails on by default — nothing to configure or maintain.

dbt Wizard is available in two surfaces:

- **In the dbt platform:** Available as a dedicated workspace and embedded in [Studio IDE](#) for development.
- **From your terminal:** [dbt Wizard CLI](#) is a terminal-native agent for local development, with or without a dbt platform account.

Learn more about [dbt Wizard](#) and its [key capabilities](#).

Why use dbt

As a dbt user, your main focus will be on writing models (select queries) that reflect core business logic – there's no need to write boilerplate code to create tables and views, or to define the order of execution of your models. Instead, dbt handles turning these models into objects in your warehouse for you.

- **No boilerplate:** Write business logic with just a SQL `select` statement or a Python DataFrame. dbt handles materialization, transactions, DDL, and schema changes.
- **Modular and reusable:** Build data models that can be referenced in subsequent work. Change a model once and the change propagates to all its dependencies, so you can

publish canonical business logic without reimplementing it.

- **Fast builds:** Use [incremental models](#) and leverage metadata to optimize long-running models.
- **Tested and documented** — Write [data quality tests](#) on your underlying data and auto-generate [documentation](#) alongside your code.
- **Software engineering workflows:** Version control, branching, pull requests, CI/CD, and [package management](#) for your data pipelines. Write DRYer code with [macros](#) and [hooks](#).
- **State-aware orchestration:** Use the dbt Fusion engine to orchestrate your dbt projects and models with [state-awareness orchestration](#), which automatically determines which models to build by detecting changes in code or data. This reduces runtime and costs by only building the models that have changed.
- **AI-powered development:** Use [dbt Wizard](#) to investigate, build, validate, and ship from natural language. dbt Wizard is grounded in your project's full context, validates its own work against lineage and tests, and includes governance and audit trails by default.

Related docs

- [Quickstarts for dbt](#)
- [Best practice guides](#)
- [What is a dbt project?](#)
- [dbt run](#)
- [dbt Wizard overview](#)
- [AI and agents](#)
- [dbt MCP server](#)

Was this page helpful?

Yes

No

[Privacy policy](#) [Create a GitHub issue](#)

This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#) apply.

 [Edit this page](#)

*Last updated on **Jun 9, 2026***